ETAS MemMapGen 12.7.0

MemMapGen Reference Guide

www.etas.com

Copyright

The data in this document may not be altered or amended without special notification from ETAS GmbH. ETAS GmbH undertakes no further obligation in relation to this document. The software described in it can only be used if the customer is in possession of a general license agreement or single license. Using and copying is only allowed in concurrence with the specifications stipulated in the contract.

Under no circumstances may any part of this document be copied, reproduced, transmitted, stored in a retrieval system or translated into another language without the express written permission of ETAS GmbH.

© Copyright 2025 ETAS GmbH, Stuttgart

The names and designations used in this document are trademarks or brands belonging to the respective owners.

ETAS MemMapGen 12.7.0 - MemMapGen Reference Guide R01 EN – 05.2025

Contents

1 Introduction . . . 4
- 1.1 Intended Use . . . 4
- 1.2 Target Group . . . 4
- 1.3 Safety Information . . . 4
- 1.4 Document Conventions . . . 4
- 1.5 Definitions and Abbreviations . . . 4

2 Invocation . . . 5
- 2.1 Introduction . . . 5
- 2.2 Command-Line Usage . . . 5
- 2.3 Output Files . . . 5
- 2.4 Redirecting Output . . . 6
- 2.5 Error and Information Messages . . . 6
- 2.6 Exit Code . . . 6

3 Command-Line Options . . . 7
- 3.1 Options Supported from MemMapGen . . . 7
- 3.2 Command Line Options . . . 9
- 3.3 Important Points for Command Line Options . . . 9
- 3.4 Sample Options File . . . 10

4 AUTOSAR MemMap Header File Generator . . . 11
- 4.1 Introduction . . . 11
- 4.2 RtaText Templates . . . 11
- 4.3 RtaText Compiler Configuration and Syntax . . . 15
- 4.4 RtaText Template Examples . . . 16
- 4.5 RtaText Syntax and Configuration for Pattern-Based MemorySection . . . 21
- 4.6 RtaText MemorySectionPattern Examples . . . 22
- 4.7 Overview of MemMapGen . . . 24
- 4.8 MemMapGen Templates Handling . . . 26

5 ETAS Contact Addresses . . . 39
- 5.1 Technical Support . . . 39
- 5.2 ETAS HQ . . . 39

1 Introduction

The MemMapGen generates the AUTOSAR memmap.h files according to the memory mapping concepts of AUTOSAR. MemMapGen takes one or more BSWMD /SWCD ARXML configuration files as input.

1.1 Intended Use

This document describes the underlying principles of MemMapGen and its features in some detail

1.2 Target Group

All ETAS RTA-CAR users that require to generate *memmap.h files for AUTOSAR components/RTE.

1.3 Safety Information

See to the RTA-CAR User Guide & ETAS_Safety_Advice.pdf supplied with the RTA-CAR installation package for additional safety information.

1.4 Document Conventions

The following conventions are used throughout this manual.

`OCI_CANTxMessage msg0`	Code snippets are shown in a monospaced Inconsola typeface.
Choose **File** > **Open**.	Menu commands are shown in boldface.
Click **OK**.	Buttons are shown in boldface.
Press <ENTER>.	Keyboard commands are shown in angled brackets.
The 'Open File' dialog box is displayed.	Names of program windows, dialog boxes, fields, etc. are shown in quotation marks.
Select the file *setup.exe*	Text in drop-down lists on the screen, program code, as well as path- and file names are shown in italics.
A *distribution* is always a one-dimensional table of sample points.	General emphasis and new terms are set in italics.

1.5 Definitions and Abbreviations

BSWMD - Basic Software Component Module Description.

SWCD - Software Component Description.

2 Invocation

2.1 Introduction

The MemMapGen generator is a Win64 executable that generates the AUTOSAR memmap.h files accordingly to the memory mapping concepts of AUTOSAR. MemMapGen takes one or more BSWMD/SWCD ARXML configuration files as input.

2.2 Command-Line Usage

The MemMapGen generator is invoked from the command line as follows:

```
MemMapGen -vm <java21.exe> --optionsfile <options_file>
or
MemMapGen -vm <java21.exe> --optionsfile [options]
```

Note: MemMapGen is used as an executable in Windows® and Linux.

Note: If MemMapGen is delivered together with RTA-CAR, java package can be found in ISOLAR-AB folder.

Command line options must be specified using long names - the supported options are listed below:

- Any number of ARXML input files can be specified.
- When a command line option takes an argument, the argument must be specified as a trailing word. For example, given an option opt with argument arg the option must be specified as `--opt arg`.
- The ordering of command line parameters is unimportant: options and ARXML files can be mixed freely. Command line options are read left-to-right and are processed before any input files are read.

See Command-line Options for a reference to all MemMapGen command-line options.

2.3 Output Files

Each execution of the MemMapGen generator creates a component MemMap header file as output.

File	Description
MemMap.h	Memory mapping header file
memmapgentool.txt	Tool log file

The following files are optional - whether they are generated or not depends on the configuration of the MemMapGen generator.

File	Description
Memmapgen_user_log.txt	User log file

2.4 Redirecting Output

Output paths must be specified for all the output options. The various output options are provided below:

As an example, the MemMapGen generator can be directed to write the MemMap header file to folder abc using the following option:

```
--pathTomemmapFileDir .\abc
```

By default, the tool log file is generated in the current directory – this is typically the same directory as the input file.

As an example, the MemMapGen generator can be directed to write the Tool log file to folder abc using the following option:

```
--armemmap_tool_log abc\MemMapGen_tool.txt
```

2.5 Error and Information Messages

The MemMapGen generator presents information on the progress of MemMap header generation using a system of status messages. Messages have the following classification:

Fatal – the detected error prevents further processing and the MemMapGen generator terminates immediately. No MemMap or associated files are generated.

Error – the detected error is serious, but it may or may not prevent further processing. MemMap and associated files are generated.

Warning - the detected error does not prevent further processing. The MemMap header and associated files will be generated but should not be considered as correct until the source of the warning has been investigated.

Information – a status message that does not indicate an error.

2.6 Exit Code

MemMapGen uses its exit status code to pass failure conditions to the calling environment:

- 0: Success – the generator completed without fatal errors. There may have been errors, warnings, or info messages.
- Non-zero: Failure – at least one fatal error occurred.

3 Command-Line Options

The operation of the MemMapGen generator is controlled via command-line options. All options begin with `--`

Parameters (i.e., filenames) specified either on the command-line or in sub-files that contain spaces must be quoted according to the rules of the invoking environment.

```
MemMapGen -vm <java21.exe> --optionsfile [options] "input filename.axml"
```

3.1 Options Supported from MemMapGen

Following MemMap configurations can be specified on the MemMapGen command-line:

Option	Value
`--arxmlFilelist`	filePath: .\arxmlFilelist.txt The path of the arxmlFilelist.txt. (This arxmlFilelist.txt file is a text file and contains the path of the input arxml files)
`--pathTomemmapFileDir`	filePath: ._gen The path of the output MemMap.h files are generated.
`--armemmap_user_log`	filePath: ._log\memmap_user.txt path of the user log file.
`--armemmap_tool_log`	filePath: ._log\memmap_tool.txt path of the tool log file.
`--platform_templates`	filePath: \platformTemplate.txt Platform templates are required to generate the MemMap headers and configure the pragma configurations for AUTOSAR memory-Sections. These files are available from Basic software.
`--customer_templates`	filePath: \customerTemplates.txt path of the customer templates.
`--armemmap_templates`	filePath: .\rtaTextTemplates.txt DSL based Platform and Customer .rtatext MemMap templates. DSL based templates are given preference over XPT templates by MemMapGen.

`--ice_ara2l_additional_platform_templates`	filePath: \additionalPlatformTemplates.txt path to additional Product–Line MemMap platform templates.
`--ice_ara2l_customer_dynamic_templates_files`	filePath: .\customerDynamicTemplates.txt path to dynamic Product–Line MemMap templates.
`--validate_start_stop_for_armemmap`	y/n If the ––validate_start_stop_for_armemmap is set as 'y', then the validation for START and STOP memory sections will be generated in the output *MemMap.h file. If the switch is set as 'n' or if it not set, then there will be no validation check generated for the START and STOP sections.
`--strict_validate_missing_pragmas`	y/n If the ––strict_validate_missing_pragmas is set to 'y', then AR memmap generator will abort with an error message if pragma configurations for the available memory sections is missing in the template files. If the switch is set as 'n' or if it is not set, then there will be no validation for pragma configurations for available memory sections.
`--machine_family`	String machine family configuration. Used to select a combination of compiler/device. Refer to the Machine Families Supported in MemMapGen (Section 4.8.3).
`--enabletimestampinlogs`	y/n This switch is used to specify if the user wants time stamps to be generated or not. If set to 'y', time stamps are generated. By default, the time stamps are disabled.

`--prj_root`	<absolute path of the project> so that absolute file paths can be built in MemMapGen using the 'absolute path of the project' and the corresponding input file relative file path. If the --prj_root is missing/invalid, then the below error message will be thrown and the tool aborts. ERROR - absolute path can't be constructed for <specified path> as parent directory does not exists/invalid. If there are only absolute paths in the supplied options file and in corresponding input file lists, then --prj_root value is not used by MemMapGen
`--list_armemmap_h_files`	filePath: .\MemMapFiles.txt Consists of the list of MemMap.h files generated with current MemMapGen tool run.
`--list_armemmap_h_files_prev`	filePath: .\MemMapFile_prev.txt Consists of the list of MemMap.h files generated with the previous run of MemMapGen tool.

3.2 Command Line Options

To display the MemMapGen product versions using long-form option names:

```
MemMapGen.exe --version
```

To display the MemMapGen help using long-form option names:

```
MemMapGen.exe --help
```

To display the MemMapGen copyright using long-form option names:

```
MemMapGen.exe --copyright
```

3.3 Important Points for Command Line Options

If there are relative paths in the options file and in corresponding input file lists, it is mandatory to supply command line argument:

```
--prj_root
```

3.4 Sample Options File

`--arxmlFilelist`	.\ArXmlFilelist.txt
`--pathTomemmapFileDir`	._gen
`--armemmap_user_ulf`	._log\armemmap_user.ulf
`--armemmap_user_log`	._log\armemmap_user.txt
`--armemmap_tool_log`	._log\armemmap_tool.txt
`--platform_templates`	.\platformTemplate.txt
`--customer_templates`	.\customerTemplate.txt
`--armemmap_templates`	.\rtaTextTemplates.txt
`--machine_family`	MCU_RB_IFX
`--validate_start_stop_for_armemmap`	y/n
`--strict_validate_missing_pragmas`	y/n
`--ice_ara2l_additional_platform_templates`	.\additionalPlatformTemplates.txt
`--ice_ara2l_customer_dynamic_templates_files`	.\DynamicTemplates.txt

4 AUTOSAR MemMap Header File Generator

4.1 Introduction

The MemMapGen header file generator generates header files containing pragma sections for the memory sections present in the BSWMD/SWCD arxml files. (Ex: RTE_BSWMD.arxml). Pragma sections can also be created for memory sections defined in the template files through extended API of memory sections.

4.1.1 Extended API for memory sections:

Extended API for memory sections: Pragma sections are generated for memory sections defined in the template files through extended API. The below API can be defined either in platform or customer templates to generate pragma sections in *MemMap.h files.

- «addMemorySection("TEST")»
- «addMemorySection("TEST_RTE","Rte")»
- The first API «addMemorySection("TEST")» generates pragma configuration for the TEST memory section in all the components *MemMap.h files.
- The second API «addMemorySection("TEST_RTE","Rte")» generates pragma configuration for the TEST memory section in the component Rte. That is Rte_MemMap.h file.

The MemMapGen file generator can be used as a standalone tool via command line.

4.2 RtaText Templates

During MemMap header file generation, the DSL based platform pragma configurations are provided through template files which have the ".rtatext" extension. These files are used to generate the respective pragma strings in MemMap header files.

Platform configurations are provided by default in RTA-CAR, however some projects require the need to override or create new configurations when the platform pragmas don't suit the project needs. In these cases the project must create customer specific pragma configurations, with the information and details to create an ".rtatext" template provided in the following sections.

RtaText Syntax and Configuration for MemorySection

RtaText syntax definition for Normal MemorySections is shown below:

```
PragmaConfiguration <Identifier>       // Parent Keyword with individual Identifier
{
    Compiler   <Selection> (Greenhills_IFX|Greenhills_ST|HighTec_IFX|Other)    // Mandatory: List of compilers
    ConfigType <Selection> (PlatformSet|CustomerPlatformSet|CustomerDynamicSet|CustomerSpecificSet)  // Mandatory

    MemorySection <Identifier>         // Multiplicity 1..N: Set Identifier conform to ShortName of SwAddrMethod
    {
        PragmaType      <Selection>     // Mandatory*: Available selection settings depend on chosen compiler
        PragmaSubType   <TextString>    // Optional: Use case/custom specific type (e.g., PORST)
        PragmaType2     <Selection>     // Optional: 2nd pragma type, specific for Greenhills compiler
        PragmaSubType2 <TextString>     // Optional: 2nd pragma sub type, specific for Greenhills compiler
        PragmaStartString <TextString> // Optional: Text based start definition, mandatory for unknown compilers
        PragmaStopString <TextString>  // Optional: Text based stop definition, mandatory for unknown compilers
        Alignments <ListOfSelection> (ALL|BOOLEAN|BIT8|BIT16|BIT32|BIT64|PTR|UNSPECIFIED)
                                        // Mandatory: For non-code sections (if PragmaType is not set to 'text'). A
  comma separated list with alignments can be set or only a single value. * In case that keywords
  PragmaStartString/PragmaStopString are not set.
    }
}
```

Figure 4.1.RtaText Syntax and Configuration for MemorySection

4.2.1 PragmaConfiguration

The PragmaConfiguration should be defined with a unique name, and only one PragmaConfiguration should be used in an RtaText template.

4.2.2 Compiler Support Options

RtaText supports four different compiler options:

- **HighTec_IFX**: HighTec compiler for IFX controllers
- **Greenhills_ST**: Greenhills compiler for ST controllers
- **Greenhills_IFX**: Greenhills compiler for IFX controllers
- **Other**: Unknown compiler

The compiler parameter can be set as shown:

```
PragmaConfiguration PlatformHightTecIFX
{
    Compiler   HighTec_IFX
    ConfigType PlatformSet
    …
}
```

The mapping of machine family and compiler configuration for RtaText is provided in the table:

Device	Compiler	Machine Family	RtaText Compiler Option
IFX	HighTec	MCU_RB_IFX/MCU_RB_IFX_UC1/MCU_R-B_IFX_UCU	HighTec_IFX
IFX	GHS	MCU_RB_IFX_GHS	Greenhill-s_IFX

ST	GHS	MCU_RB_ST/MCU_RB_ST_UCU	Greenhills_ST
NXP	GHS	MCU_RB_NXP_G22	Greenhills_ST
RH850	GHS	MCU_RB_RH850_GHS	Greenhills_ST
Other	Other	MCU_RB_GENERIC	Other

4.2.3 RtaText Templates ConfigType

RtaText templates support four different types of configurations and are listed in the order of preference:

- **CustomerSpecificSet**: Additional customer specific add-ons and to override the platform configurations. This would be a static file and is optional.
- **CustomerDynamicSet**: Additional customer dynamic configurations which can be generated based on the inputs MemorySections that are provided to MemMapGen tool. These configurations can be generated through some scripts. This is optional.
- **CustomerPlatformSet**: Additional projects specific configurations. This is optional.
- **PlatformSet**: Main platform set of pragma definitions provided by default in RTA-CAR.

Typically, the use of **CustomerPlatformSet** and **CustomerDynamicSet** is not required, however there can be scenarios where a collection of projects would require different pragmas other than PlatformSet pragmas, and when there is a need of dynamic configurations, then these ConfigTypes could be used.

The ConfigType parameter can be set as shown:

```
PragmaConfiguration PlatformHightTecIFX
{
    Compiler   HighTec_IFX
    ConfigType PlatformSet
    …
}
```

4.2.4 PragmaType Settings

The following settings can be set for PragmaType:

- bss
- data
- rodata
- sbss
- sdata
- zbss
- zdata
- text

4.2.5 PragmaSubType

Use case/custom specific type (Ex: PORST, TRAPRST, caldata)

Example:

```
MemorySection CALIB
    {
        PragmaType rodata
        PragmaSubType "caldata"
        Alignments ALL
    }
```

4.2.6 PragmaType2 Settings

The PragmaType2 is only supported in the Greenhills compiler and the following settings can be set for the PragmaType:

- bss
- data
- rodata
- sbss
- sdata
- zbss
- zdata
- text

4.2.7 PragmaSubType2

The PragmaSubType2 is only supported in the Greenhills compiler. Use case /custom specific type (Ex: PORST, TRAPRST, caldata).

Example:

```
MemorySection VAR_TRAPRST_INIT
{
    PragmaType data
    PragmaSubType "TRAPRST"
    PragmaType2 bss
    PragmaSubType2 "TRAPRST"
    Alignments ALL
}
```

4.2.8 PragmaStartString and PragmaStopString

The PragmaStartString and PragmaStopString should be used where text-based start and stop definitions are required. However the PragmaStartString & PragmaStopString shouldn't be used along with PragmaType, PragmaType2, PragmaSubType and PragmaSubType2, with this been mandatory when using unknown compilers.

4.2.9 Alignments for Non-Code Pragma Types

Specifies a list of alignments for non-code related Pragma types.

The following keywords are possible:

- **ALL**: All alignments shall be provided; It is a short term to avoid defining a list of all other keyword settings
- **BOOLEAN**: Alignment representing variables and constants of size 1 bit.
- **BIT8**: Alignment representing variables and constants of size 8 bit.
- **BIT16**: Alignment representing variables and constants of size 16 bit.
- **BIT32**: Alignment representing variables and constants of size 32 bit.
- **BIT64**: Alignment representing variables and constants of size 64 bit.
- **PTR**: Alignment representing pointers.
- **UNSPECIFIED**: Alignment representing variables, constants, structure, array, and unions where the alignment does not fit the criteria of 8, 16, 32, 64 bit or PTR

Default: If the keyword Alignments is not set, then all alignments are enabled (same as keyword is set to **ALL**), otherwise only the set list is used.

Example: **Alignments Boolean, BIT8, PTR** // Only the set alignments are selected.

In case of string-based keywords (**PragmaStartString|PragmaStopString**), a single alignment shall be set.

4.3 RtaText Compiler Configuration and Syntax

The compiler parameter can be set as shown:

```
PragmaConfiguration PlatformHighTecIFX
{
    Compiler   HighTec_IFX
    ConfigType PlatformSet
…
}
```

4.3.1 HighTec Compiler Syntax

Syntax:

#pragma section ".<PragmaType>.<SubPragmaType>.<Alignments>" <Flags>

Where:

- **PragmaType**: pragma section (text, data, bss, rodata)
- **Alignments**: Set to a power of 2 value (1,2,4,8)
- **Flags**: Set of al w | B | x | s | z attributes. optional

Definition for START-Pragma:

```
#pragma section {PragmaType}=".{PragmaType}[.{PragmaSubType}][.{DerivedAlignment}]"
 {AttributesDerivedFromPragmaType}
```

Definition for STOP-Pragma:

```
#pragma section
```

4.3.2 Greenhills Compiler Syntax

Syntax:

#pragma section <PragmaType>="PragmaType.<SubPragmaType>.<Aligments>"

or

#pragma section <PragmaType>="PragmaType.<SubPragmaType>.<Aligments>"\n #pragma section <PragmaType2>="PragmaType2.<SubPragmaType-2>.<Aligments>"

Where:

- **PragmaType** and **PragmaType2**: pragma sections (text, data, bss, rodata)
- **Alignments**: Set to a power of 2 value (1,2,4,8)

Definition for START-Pragma:

```
[#pragma startsda] Optional, only if Small Data Area pragmas are used (sdata, sbss)
[#pragma startzda] Optional, only if Zero Data Area pragmas are used (zdata, zbss)
```

When no PragmaSubType is specified:

```
#pragma ghs section {PragmaType}=default
```

When PragmaSubType is specified or Alignment is set to UNSPECIFIED:

```
#pragma ghs section {PragmaType}=".{PragmaType}[.{PragmaSubType}][.{DerivedAlignment}]"
```

Optional: Only if PragmaType2 is set and no PragmaSubType2 is specified:

```
[#pragma ghs section {PragmaType2}=default]
```

When PragmaSubType2 is specified or Alignment is UNSPECIFIED:

```
[#pragma ghs section {PragmaType2}=".{PragmaType2}[.{PragmaSubType2}][.{DerivedAlignment}]"
```

Definition for STOP-Pragma:

```
#pragma ghs section {PragmaType}="default"

[#pragma ghs section {PragmaType2}="default"]

[#pragma endsda]   Optional, only if Small Data Area pragmas are used (sdata, sbss)

[#pragma endzda]   Optional, only if Zero Data Area pragmas are used (zdata, zbss)
```

4.4 RtaText Template Examples

Examples are provided for PragmaType, PragmaSubType, PragmaType2, PragmaSubType2, PragmaStartString and PragmaStopString, with the Alignments and Flags internally handled by MemMapGen. Refer to RtaText Syntax and Configuration for MemorySection for the syntax (Section 4.2).

4.4.1 Code for MemorySection

```
MemorySection CODE
    {
```

```
        PragmaType text
    }
```

HighTec_IFX:

Start pragma section: #pragma section ".text" ax

Stop pragma section : #pragma section

Greenhills_ST/ Greenhills _IFX:

Start pragma section: #pragma ghs section text=default

Stop pragma section : #pragma ghs section text=default

4.4.2 MemorySection with PragmaType

```
MemorySection CONFIG_DATA
    {
        PragmaType rodata
        Alignments BIT8
    }
```

MemorySection:

CONFIG_DATA_8

HighTec_IFX:

Start pragma section: #pragma section ".rodata.a1" a

Stop pragma section : #pragma section

Greenhills_ST/ Greenhills _IFX:

Start pragma section: #pragma ghs section rodata=default

Stop pragma section : #pragma ghs section rodata=default

If the MemorySection CONFIG_DATA has to be configured for all the alignments, then **"ALL"** should be set for the Alignments.

```
MemorySection CONFIG_DATA
    {
        PragmaType rodata
        Alignments ALL
    }
```

MemorySection: CONFIG_DATA_8, CONFIG_DATA_16, CONFIG_DATA_32, CONFIG_DATA_64, CONFIG_DATA_BOOLEAN, CONFIG_DATA_UNSPECIFIED and CONFIG_DATA_PTR

HighTec_IFX:

Start pragma section: #pragma section ".rodata.{a1/a2/a4/a8/ax}" a

Stop pragma section : #pragma section

Greenhills_ST/ Greenhills _IFX:

Start pragma section: #pragma ghs section rodata=default

Stop pragma section : #pragma ghs section rodata=default

4.4.3 MemorySection with PragmaType and SubPragmaType

```
MemorySection CALIB
    {
        PragmaType rodata
        PragmaSubType "caldata"
        Alignments ALL
    }
```

MemorySection: CALIB_8, CALIB _16, CALIB _32, CALIB _64, CALIB _BOOLEAN, CALIB _UNSPECIFIED and CALIB _PTR

HighTec_IFX:

Start pragma section: #pragma section ".rodata.caldata.{a1/a2/a4/a8/ax}" a

Stop pragma section : #pragma section

Greenhills_ST/ Greenhills _IFX:

Start pragma section: #pragma ghs section rodata=".rodata.caldata.{a1/a2/a4/a8/ax}"

Stop pragma section : #pragma ghs section rodata=default

4.4.4 MemorySection with PragmaType and PragmaType2

PragmaType2 is supported in Greenhills_ST/Greenhills_IFX.

```
    PragmaConfiguration PlatformGreenhillsST
    {
        Compiler   Greenhills_ST
        ConfigType PlatformSet
        MemorySection VAR_INIT
    {
        PragmaType data
        PragmaType2 bss
        Alignments ALL
    }
}
```

MemorySection: VAR_INIT_8, VAR_INIT _16, VAR_INIT _32, VAR_INIT _64, VAR_INIT _BOOLEAN, VAR_INIT _UNSPECIFIED and VAR_INIT _PTR

Greenhills_ST/Greenhills_IFX:

Start pragma section: #pragma ghs section data=default \n #pragma ghs section bss=default

Stop pragma section : #pragma ghs section data=default \n #pragma ghs section bss=default

When in the case of VAR_INIT_UNSPECIFIED for Greenhills_IFX, the pragma section is different as shown below:

```
    PragmaConfiguration PlatformGreenhillsIFX
    {
    Compiler   Greenhills_IFX
    ConfigType PlatformSet
    MemorySection INTERNAL_VAR_INIT_UNSPECIFIED
    {
    PragmaStartString "#pragma ghs section data=\".data.ax\"\n  #pragma ghs section bss=\".bss.ax\""
    PragmaStopString "#pragma ghs section data=default\n  #pragma ghs section bss=default"
    }
}
```

4.4.5 MemorySection with PragmaType and SmallAddressingType

SmallAddressingType is supported for HighTec_IFX/Greenhills_IFX device.

```
MemorySection VAR_FAST_CLEARED
{
    PragmaType sbss
    Alignments ALL
}
```

MemorySection:

VAR_FAST_CLEARED_8, VAR_FAST_CLEARED _16, VAR_FAST_CLEARED _32, VAR_FAST_CLEARED _64, VAR_FAST_CLEARED _BOOLEAN, VAR_FAST_-CLEARED _UNSPECIFIED and VAR_FAST_CLEARED _PTR

HighTec_IFX:

Start pragma section: #pragma section ".sbss.{a1/a2/a4/a8/ax}" awBs

Stop pragma section : #pragma section

Greenhills_IFX:

Start pragma section: #pragma ghs startsda\n #pragma ghs section sbss=de-fault

Stop pragma section : #pragma ghs section sbss=default\n #pragma ghs endsda

When in the case of VAR_FAST_CLEARED_UNSPECIFIED for Greenhills_IFX, the pragma section is different as shown below:

```
    PragmaConfiguration PlatformGreenhillsIFX
    {
        Compiler   Greenhills_IFX
        ConfigType PlatformSet
        MemorySection VAR_FAST_CLEARED_UNSPECIFIED
    {
        PragmaStartString "#pragma ghs startsda\n  #pragma ghs section sbss=\".sbss.ax\""
        PragmaStopString "#pragma ghs section sbss=default\n  #pragma ghs endsda"
    }
}
```

4.4.6 MemorySection with PragmaType, PragmaType2 and SmallAd-dressingType

SmallAddressingType with PragmaType2 is supported for Greenhills_IFX device.

```
    PragmaConfiguration PlatformGHSIfx
    {
        Compiler   Greenhills_IFX
        ConfigType PlatformSet
        MemorySection VAR_FAST_INIT
    {
        PragmaType sdata
        PragmaType2 sbss
        Alignments BIT8,BIT16,BIT32,BIT64,PTR,BOOLEAN
    }
}
```

MemorySection:

VAR_FAST_INIT_8, VAR_FAST_INIT _16, VAR_FAST_INIT _32, VAR_FAST_INIT _64, VAR_FAST_INIT _BOOLEAN and VAR_FAST_INIT _PTR

Greenhills_IFX:

Start pragma section: #pragma ghs startsda\n #pragma ghs section sdata=default\n #pragma ghs section sbss=default

Stop pragma section : #pragma ghs section sdata=default\n #pragma ghs section sbss=default\n #pragma ghs endsda

When in the case of VAR_FAST_INIT _UNSPECIFIED, the pragma section is different as shown below:

```
MemorySection VAR_FAST_INIT_UNSPECIFIED
    {
        PragmaStartString "#pragma ghs startsda\n  #pragma ghs section sdata=\".sdata.ax\"\n  #pragma
 ghs section sbss=\".sbss.ax\""
        PragmaStopString "#pragma ghs section sdata=default\n  #pragma ghs section sbss=default\n
  #pragma ghs endsda"
    }
```

4.4.7 MemorySection with PragmaType, PragmaType2, PragmaSubType and PragmaSubType2

```
    PragmaConfiguration PlatformGHSST
    {
        Compiler   Greenhills_ST
        ConfigType PlatformSet
        MemorySection VAR_RAM_ALL_INIT
    {
        PragmaType data
                PragmaSubType "RAM_ALL_SWRESET"
                PragmaType2 bss
                PragmaSubType2 "RAM_ALL_SWRESET"
                Alignments ALL
    }
}
```

MemorySection:

VAR_RAM_ALL_INIT_8, VAR_RAM_ALL_INIT _16, VAR_RAM_ALL_INIT _32, VAR_RAM_ALL_INIT _64, VAR_RAM_ALL_INIT _BOOLEAN, VAR_RAM_ALL_INIT _UNSPECIFIED and VAR_RAM_ALL_INIT _PTR

Greenhills_ST:

Start pragma section: #pragma ghs section data=".data.RAM_ALL_SWRESET.{a1/a2/a4/a8/ax}"\n #pragma ghs section bss=".bss.RAM_ALL_SWRESET.{a1/a2/a4/a8/ax}""

Stop pragma section : #pragma ghs section data=default\n #pragma ghs section bss=default

```
    PragmaConfiguration PlatformGreenhillsIFX
    {
        Compiler   Greenhills_IFX
        ConfigType PlatformSet
        MemorySection VAR_RAM_ALL_INIT
        {
        PragmaType zdata
        PragmaSubType "RAM_ALL_DATA_SWRESET"
        PragmaType2 zbss
        PragmaSubType2 "RAM_ALL_DATA_SWRESET"
        Alignments ALL
    }
}
```

Greenhills_IFX:

Start pragma section: #pragma ghs startzda\n #pragma ghs section zdata=".zdata.RAM_ALL_DATA_SWRESET.{a1/a2/a4/a8/ax}"\n #pragma ghs section zbss=".zbss.RAM_ALL_DATA_SWRESET.{a1/a2/a4/a8/ax}"

Stop pragma section : #pragma ghs section zdata=default\n #pragma ghs section zbss=default\n #pragma ghs endzda

4.4.8 MemorySection with PragmaStartString and PragmaStopString

```
MemorySection VAR_LABEL_ONLY
{
  PragmaStartString "#pragma section \".label_only\" awB"
  PragmaStopString "#pragma section"
}
```

Start pragma section: “#pragma section “.label_only” awB”

Stop pragma section : #pragma section

4.5 RtaText Syntax and Configuration for Pattern-Based MemorySec-tion

```
PragmaConfiguration <Identifier>          // Parent Keyword with individual Identifier
{
    Compiler   <Selection>                // Mandatory: Selection of a specific compiler
    ConfigType <Selection>                // Mandatory: Selection of a specific type to determine the file content

    ComponentList <Identifier>: <ListOfTextStrings>  // Optional, Multiplicity 0..N: List of Components
                                                     // List can be placed anywhere in the file

    MemorySectionPattern <Identifier>     // Multiplicity 1..N, A unique identifier shall be set
    {
        PatternDef       <TextString>     // Mandatory: Contain definition of a pattern, e.g., "(\w+)_CODE_.*"
        ComponentListRef <Reference>      // Optional: Reference to a 'ComponentList'

        PragmaType       <Selection>      // Mandatory: Available selection settings depend on chosen compiler
        PragmaSubType    <TextString>     // Mandatory: Use case/custom specific type, referring patterns
                                          //            e.g., "swc_$1_var_init", $1 is the result of (\w+)

        PragmaType2      <Selection>      // Optional: 2nd pragma type, specific for Greenhills compiler
        PragmaSubType2   <TextString>     // Optional: 2nd pragma sub type, specific for Greenhills compiler

        // Instead of the PragmaType keywords above the following keywords can be used:
        PragmaStartString <TextString>    // Optional: Text based start definition, mandatory for unknown compilers
        PragmaStopString <TextString>     // Optional: Text based stop definition, mandatory for unknown compilers

        Alignments       <ListOfSelection>  (ALL|BOOLEAN|BIT8|BIT16|BIT32|BIT64|PTR|UNSPECIFIED)
    }                                     // Mandatory: For non-code sections, if PragmaType is not set to 'text'
}
```

Figure 4.2.RtaText Syntax and Configuration for Pattern-Based MemorySection

PragmaConfiguration, Compiler, ConfigType, PragmaType, PragmaSubType, PragmaType2, PragmaSubType2, PragmaStartString, PragmaStopString and Alignments. Refer to the sub RtaText Syntax and Configuration for MemorySec-tion for details (Section 4.2).

4.5.1 ComponentList

The Pragma configurations can be specific to BSW/ASW components, these configurations can be provided via the parameter ComponentList, with the ComponentList having one or more components.

Example:

ComponentList MyList1: “CompA”, “CompB”, “CompC”

4.5.2 PatternDef

The MemorySection can be configured by using regular expressions, with the regular expressions been configured using PatternDef.

Example:

PatternDef "(\w+)_CODE"

4.5.3 ComponentListRef

The component list which was defined using ComponentList can be referenced using the ComponentListRef.

Example:

ComponentListRef MyList1

4.6 RtaText MemorySectionPattern Examples

4.6.1 MemorySectionPattern without Regular Expression

```
    PragmaConfiguration MyConfig{
    Compiler Greenhills_IFX
    ConfigType CustomerSpecificSet
    ComponentList MyList1: "CompA", "CompB", "CompC"

    MemorySectionPattern VAR_INIT
    {
        PatternDef "VAR_INIT"
        ComponentListRef MyList1
        PragmaType data
        PragmaSubType "OS_AppX_var_init"
        Alignments BOOLEAN, BIT16, PTR
    }
}
```

VAR_INIT_BOOLEAN, VAR_INIT_16 and VAR_INIT_PTR pragma configurations are generated for CompA, CompB and CompC.

VAR_INIT_BOOLEAN:

Start pragma section: #pragma ghs section data=".data.OS_AppX_var_init.a1"

Stop pragma section : #pragma ghs section data=default

VAR_INIT_16:

Start pragma section: #pragma ghs section data=".data.OS_AppX_var_init.a2"

Stop pragma section : #pragma ghs section data=default

VAR_INIT_PTR:

Start pragma section: #pragma ghs section data=".data.OS_AppX_var_init.a4"

Stop pragma section : #pragma ghs section data=default

4.6.2 MemorySectionPattern with Regular Expression

```
    PragmaConfiguration MyConfig{
    Compiler Greenhills_ST
    ConfigType CustomerSpecificSet
    ComponentList MyList1: "CompA", "CompB", "CompC"
    MemorySectionPattern VAR_CLEARED
    {
        PatternDef "(\\w+).*_VAR_CLEARED.*"
        ComponentListRef MyList1
        PragmaType bss
        PragmaSubType "swc_$1_var_cleared"
        Alignments ALL
    }
}

<MEMORY-SECTION>
    <SHORT-NAME>CompA_VAR_CLEARED_8</SHORT-NAME>
. . .
    <SYMBOL>VAR_CLEARED_8</SYMBOL>
```

```
</MEMORY-SECTION>
```

When PatternDef consists of regular expressions, then the PatternDef is matched with the actual MemorySection.shortName from the arxml file, and the replacement strings in the pragma string is replaced with the actual group values from the regex pattern.

The PatternDef (\\w+).*_VAR_CLEARED.* is matched against MemorySection-Name CompA_VAR_CLEARED_8 and CompA is extracted and replaced in the pragma strings as shown above.

VAR_CLEARED_8:

Start pragma section: #pragma ghs section bss=".bss.swc_CompA_var_-cleared.a1"

Stop pragma section : #pragma ghs section bss=default

VAR_CLEARED_16:

Start pragma section: #pragma ghs section bss=".bss.swc_CompA_var_-cleared.a2"

Stop pragma section : #pragma ghs section bss=default

VAR_CLEARED_32:

Start pragma section: #pragma ghs section bss=".bss.swc_CompA_var_-cleared.a4"

Stop pragma section : #pragma ghs section bss=default

VAR_CLEARED_64:

Start pragma section: #pragma ghs section bss=".bss.swc_CompA_var_-cleared.a8"

Stop pragma section : #pragma ghs section bss=default

VAR_CLEARED_BOOLEAN:

Start pragma section: #pragma ghs section bss=".bss.swc_CompA_var_-cleared.a1"

Stop pragma section : #pragma ghs section bss=default

VAR_CLEARED_UNSPECIFIED:

Start pragma section: #pragma ghs section bss=".bss.swc_CompA_var_-cleared.ax"

Stop pragma section : #pragma ghs section bss=default

```
    PragmaConfiguration MyConfig{
    Compiler Greenhills_ST
    ConfigType CustomerSpecificSet
    MemorySectionPattern VAR_CLEARED
    {
        PatternDef "(\\w+).*_VAR_CLEARED.*"
        PragmaType bss
        PragmaSubType "swc_$1_var_cleared"
        Alignments ALL
    }
}
```

When the ComponetListRef is not configured in the MemorySectionPattern, then

the configuration is applicable for all the components that are passed to MemMapGen.

4.6.3 MemorySectionPattern with PragmaStartString and PragmaStop-String

When PatternDef consists of regular expressions, then the PatternDef is matched with the actual MemorySection.shortName from the arxml file. If the PatternDef is matching to the MemorySection.shortName, then for those MemorySections pragmaStartSection and PragmaStopSection will be generated with the value given in PragmaStartString and PragmaStopString:

```
    PragmaConfiguration MyConfig{
    Compiler Greenhills_ST
    ConfigType CustomerSpecificSet
    ComponentList MyList1: "CompA", "CompB", "CompC"
    MemorySectionPattern VAR_CLEARED
    {
        PatternDef "(\\w+).*_VAR_CLEARED.*"
        ComponentListRef MyList1
        PragmaStartString “#pragma ghs section
        bss=\".bss.CLSTR01_RAM_OSAPP_0_C_1_BSS_DATA_SWRESET\"”
        PragmaStopString “#pragma ghs section bss=default”
        Alignments ALL
    }
}
<MEMORY-SECTION>
<SHORT-NAME>APP_PART_VAR_CLEARED_8</SHORT-NAME>
...
<SYMBOL>VAR_CLEARED_8</SYMBOL>
</MEMORY-SECTION>
```

For the example above, the PatternDef includes the following MemorySections:

APP_PART _VAR_CLEARED_8, APP_PART _VAR_CLEARED_16, APP_PART _VAR_CLEARED_32, APP_PART _VAR_CLEARED_64, APP_PART _VAR_-CLEARED_BOOLEAN, APP_PART _VAR_CLEARED_UNSPECIFIED

The Pragma sections will be generated as:

Start pragma section: #pragma ghs section bss=\".bss.CLSTR01_RAM_OSAP-P_0_C_1_BSS_DATA_SWRESET.{a1/a2/a4/a8/ax}\"

Stop pragma section: #pragma ghs section bss=default

When the ComponetListRef is not configured in the MemorySectionPattern, then the configuration is applicable for all the components that are passed to MemMapGen:

```
PragmaConfiguration MyConfig{
Compiler Greenhills_ST
ConfigType CustomerSpecificSet
MemorySectionPattern VAR_CLEARED
{
    PatternDef "(\\w+).*_VAR_CLEARED.*"
    PragmaStartString “#pragma ghs section bss=default”
    PragmaStopString “#pragma ghs section bss=default”
    Alignments ALL
}
```

4.7 Overview of MemMapGen

The MemMapGen generator tool acts as a standalone tool. The MemMapGen tool generates the MemMap.h files (with pragma sections). The below diagram shows the MemMapGen tool diagram:

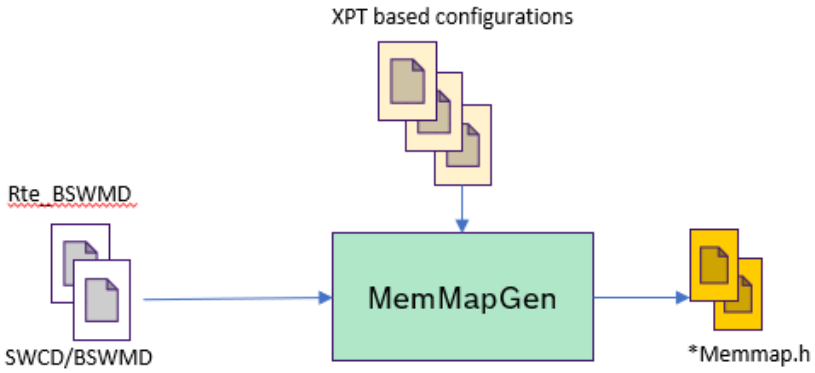

Figure 4.3.MemMapGen File Generation

The component name(shortName) for SWCD arxml files is fetched from Atomic-SwComponentType.

AtomicSwComponentType is a general type for the below SwComponentTypes:

- ApplicationSwComponentType
- ServiceProxySwComponentType
- SensorActuatorSwComponentType
- NvBlockSwComponentType
- ServiceSwComponentType
- ComplexDeviceDriverSwComponentType
- EcuAbstractionSwComponentType

4.7.1 Generating Component Name for MemMap.h Files

All the BSWMD/SWCD arxml files with same BswModuleDescription.ShortName /software-Component.shortName are grouped as one component. If the Short-Name is not present, the tool will ABORT.

<vi> This is the vendor ID of the BSW module and is an optional attribute. **<ai>** This is the vendorApiInfix of the BSW module and is an optional attribute.

If the **<vi>** and **<ai>** are present in any one of the arxml files, then the component name is generated as **ShortName_<vi>_<ai>_MemMap.h**. If only **<vi>** or **<ai>** is present in the arxml file, then it'll not be added to the component name. If the **<vi>** and **<ai>** are not defined in any one of the arxml files of same ShortName, then the component name is generated as **ShortName_MemMap.h**.

Note: Vendor ID and VendorAPIInfix isn't supported by MemMapGen for SWCD arxml files. VendorId and VendAPIInfix are expected to be available together. But

for SWCD, VendorAPIInfix won't be available.

Example:

RteComp_config_BSWMD.arxml

RteComp_BSWMD.arxml

BswModuleDescription.ShortName = Rte

<vi> = 60

<ai> = psa

Component_Name = **Rte_60_psa**

The output MemMap.h will be generated as **Rte_60_psa_Memmap.h**

4.7.2 Different Scenarios for <vi> and <ai> for same Component Name

- If any of the information missing from <vi> or <ai>.
    - If <vi> is missing, then MemMapGen generates without <vi> and <ai> information in the file name and memory allocation keywords. A warning message will be logged (AR_MEMMAP_INFO_005).
    - If <ai> is missing, then MemMapGen generates without <vi> and <ai> information in the file name and memory allocation keywords. No log messages would be logged.
- If two files having same component name (Hugo), same <ai> and different <vi>, then MemMapGen generates two MemMap.h files
    - Example: Hugo_vi1_ai_MemMap.h and Hugo_vi2_ai_MemMap.h
- If two files having same component name (Hugo) and <ai> information is available in one file and <vi> information in another file, then MemMapGen generates with-out <vi> and <ai> information in the file name and memory allocation keywords. Info message (AR_MEMMAP_INFO_001) will be logged.

The component name is case sensitive. If the same component has the short name in different case, then the tool aborts. This is applicable to vendor API also. In the above diagram the block with name "Memmap Generator" is the tool generating the MemMap files based on the Resource Consumptions files which are generally AUTOSAR files.

If only <Module>_BSWMD.arxml file contains <MEMORY-SECTIONS> then <Module>_MemMap.h is generated, otherwise corresponding component MemMap.h file is not generated. If <Component>_BSWMD.arxml file does not consist of <MEMORY-SECTIONS> but addMemorySection(memorySectionName) is defined in template files, then <Component>_MemMap.h is generated and memorysection defined in the template is added.

4.8 MemMapGen Templates Handling

4.8.1 Platform Specific Templates

Platform pragma configuration Xpand file: The snippet of IFX configuration template file is shown below:

```
«IMPORT arm»
«EXTENSION com::bosch::memmapgen::templates::MemMap_ArUtils»
«EXTENSION com::bosch::dgs::ice::armemmap::templates::MemMap_ArUtils»
«REM» Pragma Configuration«ENDREM»
 «DEFINE MCU_RB_ST_UCU_ARMEMMAP_PRAGMA_CONFIG FOR ICEArProject-»

      «configPragmaForMemSectionName("CODE").setStartMemmapPragma("#pragma ghs section text=default")»
      «configPragmaForMemSectionName("CODE").setEndMemmapPragma("#pragma ghs section text=default")»

      «configPragmaForMemSectionName("CODE_FAST").setStartMemmapPragma("#pragma ghs section text=default")»
      «configPragmaForMemSectionName("CODE_FAST").setEndMemmapPragma("#pragma ghs section text=default")»

      «configPragmaForMemSectionName("CODE_SLOW").setStartMemmapPragma("#pragma ghs section text=default")»
      «configPragmaForMemSectionName("CODE_SLOW").setEndMemmapPragma("#pragma ghs section text=default")»

      «configPragmaForMemSectionName("CALLOUT_CODE").setStartMemmapPragma("#pragma ghs section text=default")»
      «configPragmaForMemSectionName("CALLOUT_CODE").setEndMemmapPragma("#pragma ghs section text=default")»

      «configPragmaForMemSectionName("CALIB_8").setStartMemmapPragma("#pragma ghs section rodata=\".rodata.caldata.a1\"")»
      «configPragmaForMemSectionName("CALIB_8").setEndMemmapPragma("#pragma ghs section rodata=default")»

      «configPragmaForMemSectionName("CALIB_16").setStartMemmapPragma("#pragma ghs section rodata=\".rodata.caldata.a2\"")»
      «configPragmaForMemSectionName("CALIB_16").setEndMemmapPragma("#pragma ghs section rodata=default")»

      «configPragmaForMemSectionName("CALIB_32").setStartMemmapPragma("#pragma ghs section rodata=\".rodata.caldata.a4\"")»
      «configPragmaForMemSectionName("CALIB_32").setEndMemmapPragma("#pragma ghs section rodata=default")»

      «configPragmaForMemSectionName("CALIB_64").setStartMemmapPragma("#pragma ghs section rodata=\".rodata.caldata.a8\"")»
      «configPragmaForMemSectionName("CALIB_64").setEndMemmapPragma("#pragma ghs section rodata=default")»

      «configPragmaForMemSectionName("CALIB_BOOLEAN").setStartMemmapPragma("#pragma ghs section rodata=\".rodata.caldata.a1\"")»
      «configPragmaForMemSectionName("CALIB_BOOLEAN").setEndMemmapPragma("#pragma ghs section rodata=default")»
```

Figure 4.4.MemMap_ArPragmaPf_IFX.xpt Template File

4.8.2 Customer Specific Templates

Optional customer pragma configuration Xpand file: The snippet of customer IFX configuration template file is shown below:

```
«IMPORT arm»
«EXTENSION com::bosch::memmapgen::templates::MemMap_ArUtils»

«AROUND MCU_RB_ST_UCU_ARMEMMAP_PRAGMA_CONFIG FOR ICEArProject»
«targetDef.proceed()»
    «configPragmaForMemSectionName("CONST_UNSPECIFIED").setStartMemmapPragma("#pragma ghs section rodata=default")»
      «configPragmaForMemSectionName("CONST_UNSPECIFIED").setEndMemmapPragma("#pragma ghs section rodata=default")»

«configPragmaForMemSectionName("CONST_SAVED_RECOVERY_ZONE_UNSPECIFIED").setStartMemmapPragma("#pragma ghs section rodata=default")»
      «configPragmaForMemSectionName("CONST_SAVED_RECOVERY_ZONE_UNSPECIFIED").setEndMemmapPragma("#pragma ghs section rodata=default")»

«ENDAROUND»
```

Figure 4.5.Sample Customer Template IFX xpt Template File

The snippet of customer IFX configuration template file in which the list of components or a single component is passed through the template is shown below:

```
«LET {"RTE3","RTE4","RTE5"} AS listOfComponents»
    «configPragmaForMemSectionName("VAR_8BIT", listOfComponents).setStartMemmapPragma("#pragma section \".cust.var8\" aw")»
    «configPragmaForMemSectionName("VAR_8BIT", listOfComponents).setEndMemmapPragma("#pragma section")»
    «ENDLET»
```

Figure 4.6.List of Components

```
«configPragmaForMemSectionName("VAR_8BIT", "RTE").setStartMemmapPragma("#pragma section \".cust.var8\" aw")»
    «configPragmaForMemSectionName("VAR_8BIT", "RTE").setEndMemmapPragma("#pragma section")»
```

Figure 4.7.Single Component

The snippet of customer IFX configuration template file in which the memory section name has regex pattern, and the pragma string has replacement strings in it is shown below:

```
«configPragmaForMemSectionName("EcuP_OsApp[(a-zA-Z)+].*", "RTE").setStartMemmapPragma("#pragma section \".text.partition_$1\"")»
    «configPragmaForMemSectionName("EcuP_OsApp[(a-zA-Z)+].*", "RTE").setEndMemmapPragma("#pragma section")»
    «LET {"RTE3","RTE4","RTE5"} AS listOfComponents»
    «configPragmaForMemSectionName("EcuP_OsApp(\\d+).*", listOfComponents).setStartMemmapPragma("#pragma section \".text.partition_$1\"")»
    «configPragmaForMemSectionName("EcuP_OsApp(\\d+).*", listOfComponents).setEndMemmapPragma("#pragma section")»
    «ENDLET»
```

Figure 4.8.Regex in Memory Section Name with List of Components

In case the memory section name in the customer template has regex in it, then the pattern is matched with the actual memory section name from the arxml file, and the replacement strings in the pragma string is replaced with the actual group values from the regex pattern.

```
</MEMORY-SECTION>
      <MEMORY-SECTION>
    <SHORT-NAME>EcuP_OsAppABC_CODE</SHORT-NAME>
      <SW-ADDRMETHOD-REF BASE="Rte_BSWMD_SwAddrMethods" DEST="SW-ADDR-METHOD">VAR_8BIT</SW-ADDRMETHOD-REF>
    <ALIGNMENT>UNSPECIFIED</ALIGNMENT>
</MEMORY-SECTION>
```

Customer template containing regex

```
      «configPragmaForMemSectionName("EcuP_OsApp[(a-zA-Z)+].*", "RTE1").setStartMemmapPragma("#pragma section \".text.partition_$1\"")»
      «configPragmaForMemSectionName("EcuP_OsApp[(a-zA-Z)+].*", "RTE1").setEndMemmapPragma("#pragma section")»
```

Output memmap.h file content

```
#elif defined RTE1_START_SEC_EcuP_OsAppABC_CODE
#pragma section ".data.ReIni" aw
#undef RTE1_START_SEC_EcuP_OsAppABC_CODE
#elif defined RTE1_STOP_SEC_EcuP_OsAppABC_CODE
#pragma section
#undef RTE1_STOP_SEC_EcuP_OsAppABC_CODE
```

Figure 4.9.Regex Replacement Example

If the number of groups in the regex pattern is lesser than the number of replacement strings in the replacement pattern, then a warning message with ID **AR_MEMMAP_WARN_012** is logged. Because of this, no pragma string would be generated for the configured MemAllocKeyword in the *.MemMap.h file. If any code is using it, then it would lead to compiler default pragma.

Customer template containing regex

```
      «configPragmaForMemSectionName("EcuP_OsApp[(a-zA-Z)+].*", "RTE1").setStartMemmapPragma("#pragma section \".text.partition_$1_$2\"")»
      «configPragmaForMemSectionName("EcuP_OsApp[(a-zA-Z)+].*", "RTE1").setEndMemmapPragma("#pragma section")»
```

armemmap_txt file content will have the warning message

```
Number of groups in the pattern "{{pattern}}" does not match the number of $x(replacement string) in replacement pattern
```

Figure 4.10.Regex Pattern Group and Replacement String in Replacement Pattern Mismatch

4.8.3 Machine Families Supported in MemMapGen

MemMapGen supports the following combination of devices/compilers as shown in the below table:

Device/Compiler	Hightec
IFX	MCU_RB_IFX/MCU_RB_IFX_UC1/MCU_RB_IFX_UCU
Device/Compiler	**GHS**
IFX	MCU_RB_IFX_GHS
NXP	MCU_RB_NXP_G22
ST	MCU_RB_ST/MCU_RB_ST_UCU
RH850	MCU_RB_RH850_GHS

4.8.4 Generic Machine Family

If the use of a customer specific device and compiler is required, then the "MCU_RB_GENERIC" must be specified as the machine family in the project configuration.

Pragma configurations for the generic machine family is not available in the platform template. Therefore these must be configured in the customer template.

The sample customer template is shown below.

```
«IMPORT arm»
«EXTENSION com::bosch::memmapgen::templates::MemMap_ArUtils»

«AROUND MCU_RB_GENERIC_ARMEMMAP_PRAGMA_CONFIG FOR ICEArProject»
«targetDef.proceed()»

    «configPragmaForMemSectionName("CODE").setStartMemmapPragma("#pragma section \".code.cust\" aw")»
    «configPragmaForMemSectionName("CODE").setEndMemmapPragma("#pragma section")»

«ENDAROUND»
```

Figure 4.11.Customer Template Sample for Generic Machine Family

4.8.5 Default Machine Family in RTA-CAR: MCU_RB_DEFAULT

The MCU_RB_DEFAULT machine family option is selected by default in the RTA-CAR project creation UI. This is selected to ensure that the output of MemMapGen is compatible with any compiler/hardware combination.

The tradeoff is that this default option shall not perform any memory section mapping. Instead, to use this by default, the project must use the template integration file "Bsw_MemMap.h" to define the mapping.

In AUTOSAR this mapping is done on a per module basis, RTA-BSW reduces the number of mappings by defining a single "BSW" namespace used by each type of data.

The example shown below is from CanSM where CANSM_START_SEC_CONST_8 becomes BSW_START_SEC_CONST_8, any other modules using "CONST_8" will use the same definition: BSW_START_SEC_CONST_8.

```
#if defined CANSM_START_SEC_CONST_8
  #define BSW_START_SEC_CONST_8
  #include "Bsw_MemMap.h"
  /* MR12 RULE 20.5 VIOLATION: AUTOSAR MemMap concept requires #undef, AUTOSAR MemMap requirements are incompatible to MISRA */
  #undef CANSM_START_SEC_CONST_8
#elif defined CANSM_STOP_SEC_CONST_8
  #define BSW_STOP_SEC_CONST_8
  #include "Bsw_MemMap.h"
  /* MR12 RULE 20.5 VIOLATION: AUTOSAR MemMap concept requires #undef, AUTOSAR MemMap requirements are incompatible to MISRA */
  #undef CANSM_STOP_SEC_CONST_8
```

Figure 4.12.Sample MemMap.h Content with the Inclusion of BSW Macros

Note: This default option is provided for flexibility and to support pre-MemMapGen users who already manually mapped the BSW using "Bsw_MemMap.h".

However, new users are encouraged to take full advantage of the MemMapGen tool by choosing the supported machine family arguments for the compiler/hardware combination. See Machine Families Supported in MemMapGen (Section 4.8.3), or if the combination is currently unsupported, using the MapMapGen script interface to automatically map the memory sections to your specification.

4.8.6 Pragma Configuration in Template Files

There can be four types of pragma configurations in the xpt files:

- Default pragma configuration:
- Pragma configuration for memory section name (non De–serap):
- Pragma configuration for complete address method name (De–Serap):
- Pragma configuration for address method name starting with specified string (De-Serap):abbreviation:

4.8.7 Mapping of Memory Elements in the MemMap Files

One MemMap.h file is generated for each component of RTE. All the memorySections present in the component will have corresponding pragma section in the MemMap.h.

Algorithm for generating pragma sections of a memorySection:

- If a memory element name having a corresponding configuration «configPragmaForMemSec-tionName» in xpt file, then pragma section is taken from this configuration, otherwise:
- If the address of the memory element is having corresponding configuration «configPrag-maForAds» in xpt file, then pragma section is taken from this configuration, otherwise:
- If the address of the memory element is having corresponding configuration «configPrag-maForAds» which starts with string* in xpt file, then pragma section is taken from this configuration, otherwise:
- Default pragma section is taken from the configuration «configPragmaDefault()» otherwise:

- If no pragma configuration is available for the memorySection in the template file, then the be-low comment is generated in the MemMap.h file

/* VAR_UNSPECIFIED pragma configuration not available, please update the respective template file */ where VAR_UNSPECIFIED is the memorySection.shortName or is combination of prefixRef and symbol. Additionally, AR_MEMMAP_WARN_006 would be logged in the MemMapGen logs.

Example:

ICE Message ID: AR_MEMMAP_WARN_006, WARN: Pragma configuration couldn't be found for the <MEMORY-SECTION> 'SAMPLE' present in the arxml file inputRte_BSWMD.arxml.

The Memmap section looks like below in the MemMap.h file:

```
#if defined RTE2_START_SEC_VAR_8BIT
    #pragma section .data.ReIni aw
    #undef RTE2_START_SEC_VAR_8BIT
    #undef MEMMAP_ERROR
    #elif defined RTE2_STOP_SEC_VAR_8BIT
    #pragma section
    #undef RTE2_STOP_SEC_VAR_8BIT
    #undef MEMMAP_ERROR
    #endif
```

The MemMap file name is generated by using the RTE component file name.

Example: If the RTE component name is <swcName>{_<vi>_<ai>}, then the name of the MemMap file is **swcName_<vi>_<ai>memmap.h**.

4.8.8 Support of MemoryAllocationKeyword for ASW Components

MemMapGen supports ASW components that are present in the Rte_BSWMD.arxml file which is generated by RTE. If the memory section name appears in camel case, then the MemAllocKeyword would be generated in camel case in Rte_MemMap.h file.

The memory section with the respective pragma section should be configured in the customer templates otherwise MemoryAllocationKeyword will be generated without pragma section in the Rte_MemMap.h.

Example:

```
Short Name of BswModuleDescription - Rte
<MEMORY-SECTION>
<SHORT-NAME>SlowNoReIni</SHORT-NAME>
<ALIGNMENT>8</ALIGNMENT>
<SW-ADDRMETHOD-REF BASE="Rte_MCSD_SwAddrMethods" DEST="SW-ADDR-METHOD">SlowNoReIni</SW-ADDRMETHOD-REF>
<SYMBOL>SlowNoReIni_Symbol</SYMBOL>
</MEMORY-SECTION>
```

MemAllocKeyword in Rte_MemMap.h:

```
RTE_START_SEC_SlowNoReIni_Symbol
RTE_STOP_SEC_SlowNoReIni_Symbol
```

4.8.9 Support for ASW (SWCD files)

MemMapGen supports generation of *_MemMap.h files for MemorySections from SWCD arxml files. The MemMap file is generated by adding shortName from softwareComponent. Mem-MapGen would read the MemorySections which are part

of SwcImplementation and generate them in the MemMap files. The file name is generated as follows – softwareCompo-nent.shortName_MemMap.h.

4.8.10 Generation of Memory Allocation Keywords

Usually, memory allocation keywords are generated using the BswModuleSecription.shortName and Memorysection.shortName **softwareComponent.shortName_START_SEC_memorySection.shortName**.

If <SYMBOL> information is found inside <MEMORY-SECTION>, then it would be used for generation of memory allocation keyword in Rte_MemMap.h. It would be generated as shown below

softwareComponent.shortName_START_SEC_memorySection.symbol

softwareComponent.shortName_STOP_SEC_memorySection.symbol

If **<PREFIX-REF>** and **<SYMBOL>** information is found inside <MEMORY-SECTION>. Then it would be used for generation of memory allocation keyword in MemMap.h file. It would be generated as shown below:

PREFIX-REF_START_SEC_<SYMBOL>

PREFIX-REF_STOP_SEC_<SYMBOL>

Where **PREFIX-REF** is the Prefix-Ref referred from **<SECTION-NAME-PREFIX>.SYMBOL**

Note: softwareComponent name is considered case sensitive during MemAllocKeyword gener-ation in case of SwcImplementation.

4.8.11 Generation of Memory Allocation Keywords Based on Prefix-Ref and Symbol

Usually, memory allocation keywords are generated using the BswModuleDescription.shortName and Memorysection.shortName:

BswModuleDescription.Shortname_[vi_ai]_START_SEC_MemorySection.shortName

If <SYMBOL> information is found inside <MEMORY-SECTION>, then it would be used for generation of memory allocation keyword in Rte_MemMap.h. It would be generated as shown below:

softwareComponent.shortName_START_SEC_memorySection.symbol

softwareComponent.shortName_STOP_SEC_memorySection.symbol

If **<PREFIX-REF>** and **<SYMBOL>** information is found inside <MEMORY-SECTION>. Then it would be used for generation of memory allocation keyword in MemMap.h file. It would be generated as shown below:

PREFIX-REF_START_SEC_<SYMBOL> PREFIX-REF_STOP_SEC_<SYMBOL>

Where **PREFIX-REF** is the Prefix-Ref referred from **<SECTION-NAME-PREFIX>.SYMBOL**

Example:

RteBSWMD.arxml:

```
<MEMORY-SECTION>
<SHORT-NAME>RTE_EXT_CODE</SHORT-NAME>
<PREFIX-REF BASE="Rte_BSWMD_BswImplementations" DEST="SECTION-NAME-PREFIX">Rte/ResConsumption/RTE_EXT<
/PREFIX-REF> <SW-ADDRMETHOD-REF BASE="Rte_BSWMD_SwAddrMethods" DEST="SW-ADDR-METHOD">CODE<
/SW-ADDRMETHOD-REF>
<SYMBOL>CODE</SYMBOL>
</MEMORY-SECTION>
<SECTION-NAME-PREFIX>
<SHORT-NAME>RTE_EXT</SHORT-NAME>
<SYMBOL>RTE_EXT</SYMBOL>
</MEMORY-SECTION>
```

MemAllocKeyword in Rte_MemMap.h:

RTE_EXT_START_SEC_CODE

RTE_EXT_STOP_SEC_CODE

MemMapGen would validate if the Prefix-Ref starts with the component name, if it doesn't start then tool wouldn't generate the Memory-Section based on Prefix-Ref in Rte_MemMap.h file and a low priority warning would be logged.

If Prefix-Ref consists of camel case, then MemMapGen would generate the memory allocation keywords in upper case only. RTE would generate the Prefix-Ref always in upper case only.

If the Prefix-Ref does not contain a symbol, the MemMapGen aborts with an error message – AR_MEMMAP_ERROR_003: <SYMBOL> tag is missing in <SECTION-NAME-PREFIX> "rba_IoExtFscDosVlv". SYMBOL is a mandatory parameter for SECTION-NAME-PREFIX.

4.8.12 Generation of Validation Check for START and STOP Sections

An option validate_start_stop_for_armemmap has been supported in the options file for generating the validation check for START and STOP sections. Allowed values for the switch are "y/n". Only when the switch is set to "Y" in the options file, the validation check will be generated in the output MemMap.h file.

This Validation check is required to verify that after a START section the corresponding STOP section is called and not some other START or STOP. This is re-quired for CUBAS MemMap Handling.

The MISRA comments and the machine family type are generated in the MemMap.h file along with the MemMapGen tool version. This is irrespective of the switch.

The machine family name or the machine type is fetched from the option **machine_family** Misra comment:

/* MR12 DIR 4.10 VIOLATION: Memmap file must be included multiple times */

/* MR12 RULE 20.5 VIOLATION: AUTOSAR MemMap concept requires #undef, AUTOSAR Mem-Map requirements are incompatible to M- ISRA */

An example containing the MISRA comments, machine type/machine family name and Valida-tion check for START and STOP sections is available below.

```
/*
 ****************************************************************************************************************
 *
 * COPYRIGHT RESERVED, Robert Bosch GmbH, 2019. All rights reserved.
 * The reproduction, distribution and utilization of this document as well as the communication of its contents to
 * others without explicit authorization is prohibited. Offenders will be held liable for the payment of damages.
 * All rights reserved in the event of the grant of a patent, utility model or design.
 *
 ****************************************************************************************************************
 ****************************************************************************************************************
 Machine Type: MCU_RB_IFX_UC1
 Generated by ICE 2023
 ****************************************************************************************************************
*/
/* MR12 DIR 4.10 VIOLATION: Memmap file must be included multiple times */
/* MR12 RULE 20.5 VIOLATION: AUTOSAR MemMap concept requires #undef, AUTOSAR MemMap requirements are incompatible to MISRA */
```

Figure 4.13.Misra Warning with machineType

```
#elif defined RTE_START_SEC_CODE
  #ifdef RTE_SEC_STARTED
    #error "Missing STOP section for previously started section"
  #else
    #define RTE_SEC_STARTED
  #endif
  #ifdef RTE_START_SEC_CODE_STARTED
    #error "Section START_SEC_CODE defined redundantly"
  #else
    #define RTE_START_SEC_CODE_STARTED
  #endif
        #pragma section ".text" ax
        #undef RTE_START_SEC_CODE
#elif defined RTE_STOP_SEC_CODE
        #ifdef RTE_SEC_STARTED
                #undef RTE_SEC_STARTED
                #ifndef RTE_START_SEC_CODE_STARTED
                        #error "Missing STOP section for previously started section"
                #endif
        #endif
        #ifdef RTE_START_SEC_CODE_STARTED
                #undef RTE_START_SEC_CODE_STARTED
        #else
                #error "Missing section START_SEC_CODE"
        #endif
        #pragma section
        #undef RTE_STOP_SEC_CODE
```

Figure 4.14.Start and Stop Validation Check

4.8.13 Strict Validation Check for Missing Pragma Configurations

An option strict_validate_missing_pragmas has been supported in the options file of MemmapGen for validation of missing pragma configurations for available memory sections in the template files.

This validation check helps in verifying that all the available memory sections have pragma configurations in the template files.

This is required for CUBAS MemMap Handling. Allowed values for the switch are “y /n”. Only when the switch is set to “y” in the options file, and pragma configurations are missing for the available memory sections in the templates then the tool aborts with error message **AR_MEMMAP_ERROR_004**.

4.8.14 Cucel.MemMap, Additional and Dynamic Templates

- MemMapGen tool maintains platform templates through Cucel.MemMap. They are updated and released as part BSW software.
- Product–Line specific templates can consist of some platform memory sections which can be maintained separately.
- Additional Product–Line templates are provided via the argument "ice_ara2l_additional_platform_templates".
- When two set of MemMap templates and customer template is provided to MemMap gen-eration, then the preference is as follows:
    1. Customer Templates (Priority 1).
    2. Product–Line platform templates (Priority 2).
    3. MemMap platform templates (Priority 3).

In some cases, Dynamic Product–Line templates are provided via the argument "ice_ara2l_customer_dynamic_templates_files". When three set of MemMap templates and customer template is provided to MemMap generation, then the preference is as follows:

1. Customer Templates (Priority 1).
2. Dynamic Product–Line platform templates (Priority 2).
3. Product–Line platform templates (Priority 3).
4. MemMap platform templates (Priority 4).

4.8.15 MemMapGen Workflow

The workflow below provides the information of how the pragma configurations are fetched in MemMapgen.

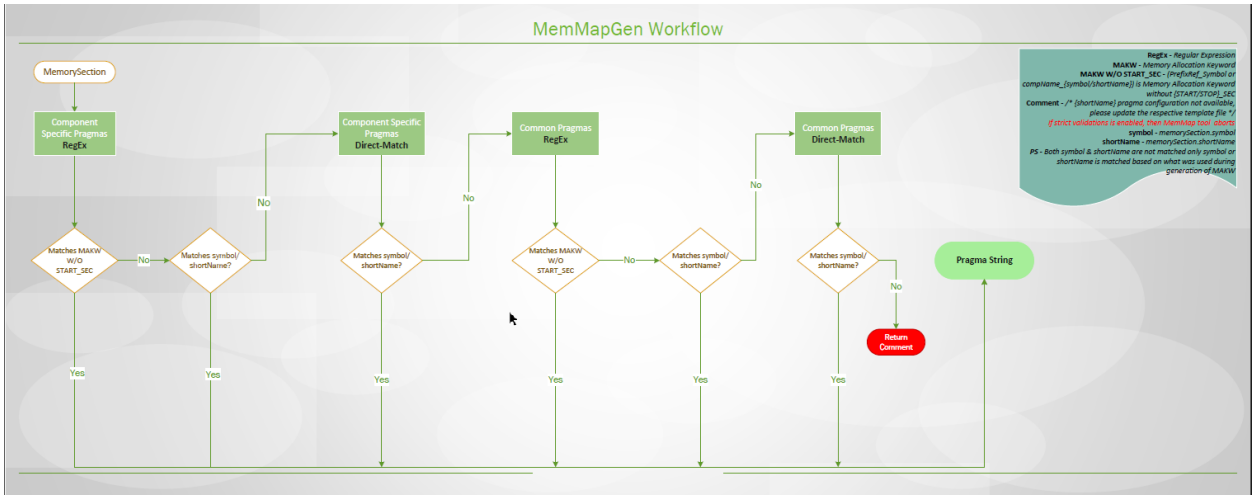

Figure 4.15.MemMapGen Workflow

5 ETAS Contact Addresses

5.1 Technical Support

Technical support is available to all users with a valid support contract. If you do not have a valid support contract, please contact your regional sales office.

- https://www.etas.com/en/contact.php

The best way to get technical support is using the online support portal RTA Hotline:

- https://rtahotline.etas.com/

It is helpful if you can provide technical support with the following information (where relevant):

- Your support contract number
- The version of the ETAS tools you are using
- The version of the compiler tool chain you are using
- The command line (or reproduction of steps) that result in an error message
- The error messages or return codes you received (if any)
- Your RTA-CAR Project or if not possible the arxml file(s) containing the error(s).
- The *Diagnostic.dmp* if it was generated

5.2 ETAS HQ

ETAS GmbH Borsigstraße 24 70469 Stuttgart Germany	Phone:	+49 711 3423-0
	Fax:	+49 711 3423-2106
	WWW:	www.etas.com